

Modelos Generativos Profundos

Clase 8: Modelos autorregresivos y LLMs (parte IV)

Fernando Fêtis Riquelme

Primavera, 2025

Facultad de Ciencias Físicas y Matemáticas

Universidad de Chile

Otros modelos tipo Transformer

Large language models

Otros modelos tipo Transformer

BERT | bidirectional encoder representations from transformers

BERT es un modelo tipo Transformer usado para tareas de comprensión del lenguaje.

- La entrada al modelo puede ser una secuencia de tokens $S \in \mathcal{V}^*$ o dos secuencias de tokens $S_1, S_2 \in \mathcal{V}^*$ concatenadas.
- No sigue un enfoque autorregresivo, por lo que no necesita masking causal y no puede generar texto.
- Los embeddings contextuales obtenidos con BERT pueden ser usados para otras tareas. También es posible obtener un embedding representante de toda la secuencia de entrada.
- BERT utiliza 3 tokens especiales: **<MASK>**, **<CLS>** y **<SEP>**.

BERT | tokenización

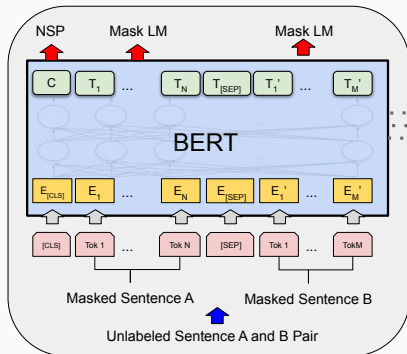
- BERT añade el token <CLS> al comienzo. Su embedding contextual es usado como la representación agregada de la entrada.
- Las secuencias $S_1, S_2 \in \mathcal{V}^*$ concatenadas son separadas por el token <SEP>, y cada secuencia incluye un embedding adicional.

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	$E_{[\text{CLS}]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[\text{SEP}]}$	E_{he}	E_{likes}	E_{play}	$E_{\text{##ing}}$	$E_{[\text{SEP}]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

BERT | entrenamiento

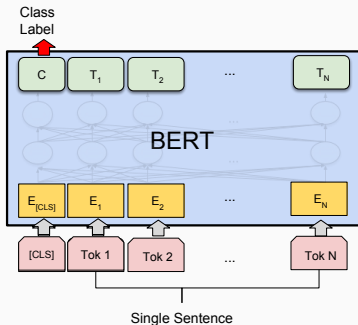
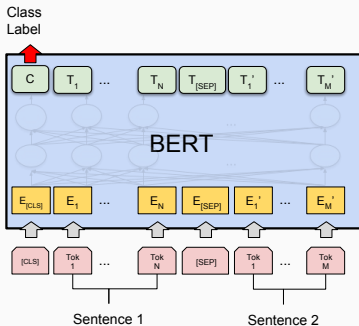
BERT se entrena mediante dos tareas autosupervisadas:

- **Masked language model:** se ocultan tokens en la secuencia de entrada (se cambian por <MASK>) y el modelo debe predecirlas.
- **Next sentence prediction:** el modelo debe predecir si el texto $S_2 \in \mathcal{V}^*$ es la continuación del texto $S_1 \in \mathcal{V}^*$.



BERT | uso

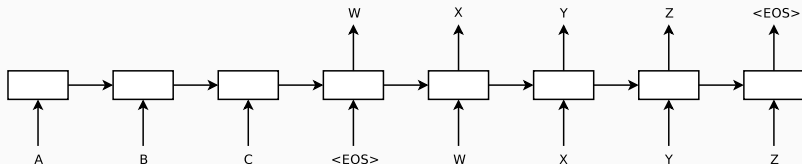
- Una vez entrenado, BERT puede ser usado para clasificar secuencias individuales o pares de secuencias. Para esto utiliza el embedding contextual del token <CLS>.
- Es necesario cambiar el cabezal de clasificación original por otro cabezal específico para la tarea y hacer **fine tuning**.



Arquitecturas encoder-decoder | modelo seq2seq

Si se tiene un modelo autorregresivo condicional $p(S|S_{\text{contexto}})$, donde $S_{\text{contexto}} \in \mathcal{V}^*$ es una secuencia condicional, entonces es posible procesar ambas secuencias por modelos diferentes:

- Un **encoder** procesa S_{contexto} y genera una representación contextual para S_{contexto} .
- Un **decoder** genera autorregresivamente la secuencia $S \in \mathcal{V}^*$ utilizando la representación contextual obtenida con el encoder.

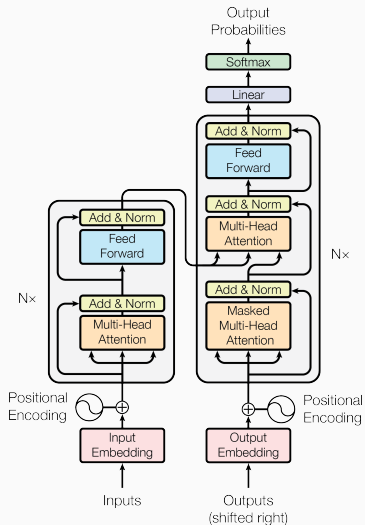


El modelo **encoder-decoder**, también llamado **seq2seq**, fue modelado originalmente usando RNNs. Sin embargo, también se puede utilizar una arquitectura **Transformer**:

- Un encoder tipo BERT procesa todo el contexto de entrada S_{contexto} .
- Un decoder tipo GPT genera la secuencia S utilizando la representación contextual obtenida con el encoder.
- Para esto, el decoder agrega un bloque de **atención cruzada** hacia las representaciones contextuales del encoder.

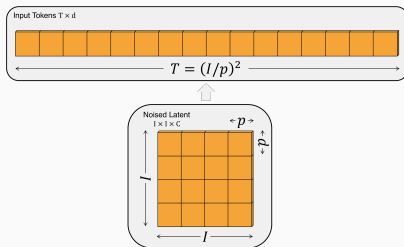
Arquitecturas encoder-decoder | arquitectura Transformer

- GPT y BERT son arquitecturas derivadas del Transformer original.
- Hoy en día la mayoría de los modelos son de tipo GPT (**decoder-only**).
- Modelos encoder-decoder como T5 siguen siendo ampliamente utilizados en modelos text-to-image.
- Los Transformers también tienen propiedades de aproximación universal y son turing-completos.



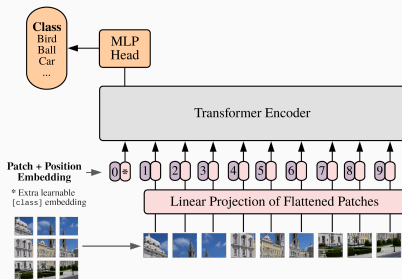
Vision Transformer | patchificación

- BERT es un buen clasificador de secuencias, lo que motiva a transformar una imagen en una secuencia para usar BERT.
- La atención tiene costo cuadrático, por lo que no se podrían clasificar imágenes de alta resolución si se trata cada pixel como un token embedding de dimensión $I \times I \times C$.
- **Vision Transformer (ViT)** propone dividir la imagen en **patches**, donde cada patch aplanado se trata como un token.



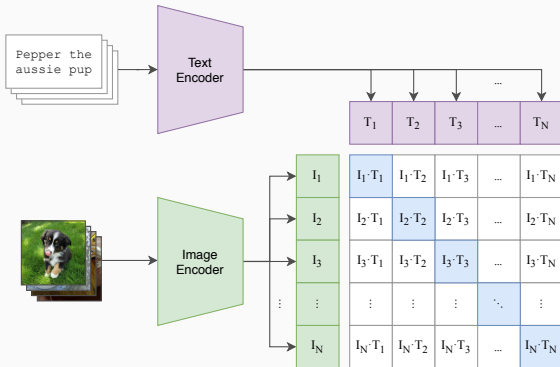
Vision Transformer | clasificación

- Al igual que en BERT, se agrega un token especial <CLS> cuyo embedding se utiliza para la clasificación.
- Una vez entrenado, se puede sustituir el cabezal de clasificación por otro y hacer fine tuning.
- ViT en general funciona mejor que una CNN pero requiere una gran cantidad de datos para su entrenamiento.



CLIP | contrastive language-image pretraining

- Se puede usar un image encoder (p.g. ViT) y un text encoder (p.g. BERT) para obtener representaciones de imágenes y texto en un mismo espacio de embedding.
- **CLIP** logra esto utilizando un **enfoque contrastivo**. En particular, CLIP no es un modelo generativo.



Si $\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$, es un conjunto de pares de imágenes $x_n \in \mathbb{R}^{D_{\text{imagen}}}$ con sus respectivos textos descriptivos $t_n \in \mathcal{V}^*$:

- Se calculan los vectores de embedding $\text{ImageEncoder}(x_n) \in \mathbb{R}^{D_I}$ y $\text{TextEncoder}(t_n) \in \mathbb{R}^{D_T}$ para cada instancia $(x_n, t_n) \in \mathcal{D}$.
- Estos embeddings son proyectados a un espacio común, $\mathbb{R}^D$, para obtener nuevos embeddings $I_n, T_n \in \mathbb{R}^D$.
- De este modo, CLIP se entrena maximizando

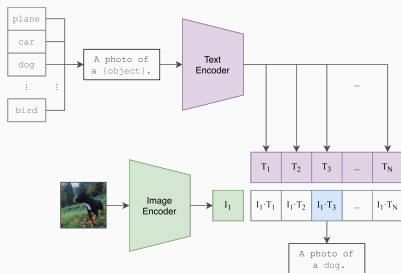
$$\sum_{i=1}^N \log \left(\frac{\exp(L_{ii})}{\sum_{j=1}^N \exp(L_{ij})} \right) + \sum_{j=1}^N \log \left(\frac{\exp(L_{jj})}{\sum_{i=1}^N \exp(L_{ij})} \right),$$

donde $L_{ij} = s_\theta \cdot \text{SimilitudCoseno}(I_i, T_j) \in \mathbb{R}$, con $s_\theta > 0$ un parámetro de temperatura (inversa) entrenable.

CLIP | usos

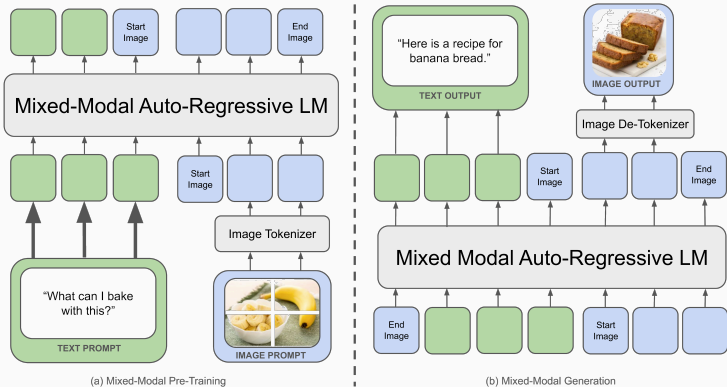
- Una vez entrenado, se puede comparar la similitud entre una imagen y un texto mediante la similitud coseno de sus respectivos vectores de embedding (proyectados al espacio común).
- Se puede usar CLIP como un **open-vocabulary classifier** comparando una imagen con un conjunto de textos de la forma

$S_{<CLASE>} = \text{"una imagen de un/una } <CLASE>."$



Chameleon

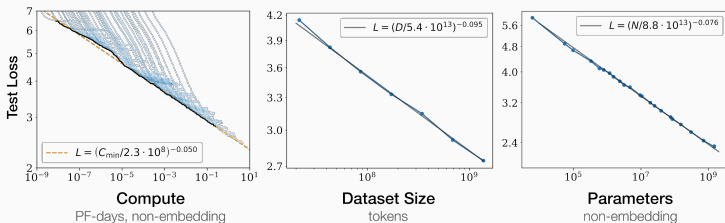
El modelo **Chameleon** utiliza la técnica de patchificación de imágenes para entrenar un modelo autorregresivo tipo GPT para generar texto e imágenes de manera conjunta.



Large language models

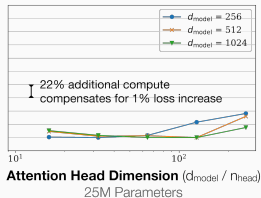
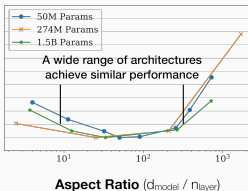
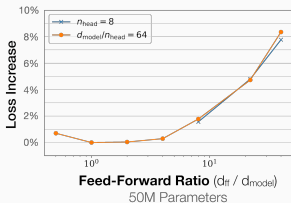
Leyes de escala

- El entrenamiento auto-supervisado permite escalar fácilmente un LLM ya que es fácil obtener datos de entrenamiento.
- Un LLM se puede escalar en el **número de parámetros**, en la **cantidad de datos** de entrenamiento y en la **cantidad de cómputo** usada para entrenar.
- Se ha demostrado empíricamente que la loss de un LLM sigue algunas **leyes de escala** de la forma $y = ax^{-p}$, con $a, p > 0$.
- En escala logarítmica, $\tilde{y} = \log(ax^{-p}) = -p\tilde{x} + \log a$ es una recta.



Leyes de escala

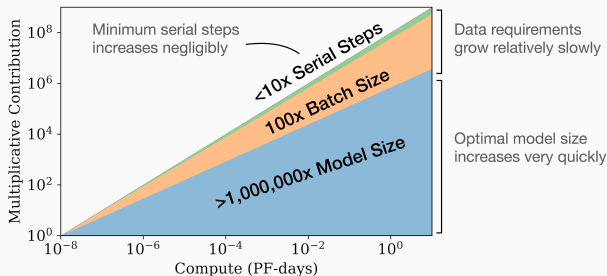
- Las leyes de escala se han verificado empíricamente en un amplio rango de parámetros y en diversos datasets de entrenamiento.
- Estas leyes permiten estimar el rendimiento de un entrenamiento mucho más costoso, lo que entrega confianza para escalar.
- El rendimiento de un LLM no depende fuertemente de algunos hiperparámetros de arquitectura (**the bitter lesson**).



- Estos patrones permiten ajustar hiperparámetros en modelos pequeños y luego extrapolarlos a modelos más grandes, donde solo se puede entrenar una única vez.

Leyes de escala

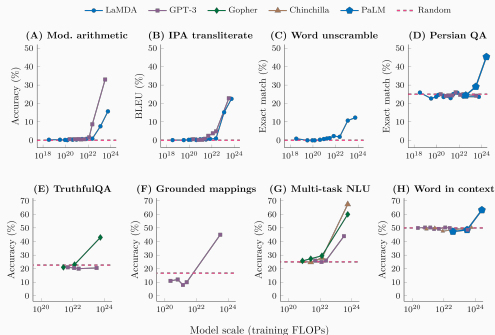
- No todos los parámetros de entrenamiento deben ser escalados de la misma manera.



- **Chinchilla scaling laws:** dado un presupuesto de cómputo, permiten obtener la asignación de recursos óptima.
- En particular, modelos más grandes entrenados menos tiempo pueden funcionar mejor que modelos más pequeños entrenados hasta la convergencia.

Habilidades emergentes

- Una habilidad se dice **emergente** si no está presente en modelos más pequeños, sino que aparece de manera repentina en modelos más grandes.
- Por definición, estas habilidades no son predecibles entrenando modelos más pequeños.



Prompting

- Una propiedad de los LLMs es su capacidad de seguir patrones. Por ejemplo, para una entrada $S \in \mathcal{V}^*$ de la forma

$S = \text{"Q: ¿Cuál es la capital de Francia? R: "},$

el LLM podría continuar la generación con "París" (un ARM no distingue entre la entrada y lo que ha generado).

- Para comportamientos más complejos (p.g. **modelos conversacionales**), se suele realizar fine tuning y usar **templates**.
- Del mismo modo se puede adaptar un LLM para **seguir instrucciones**, las cuales se indican en el **prompt**.
- El prompting puede ser muy efectivo para guiar el comportamiento de un modelo (p.g. **chain of thought (CoT)** o **ReAct**).

En la próxima clase.

- **Propiedades agénticas:** RAG, uso de herramientas, ReAct.
- **Aspectos de optimización:** uso de memoria, implementaciones eficientes, heurísticas de entrenamiento.
- **Fine tuning:** instruction fine tuning y LoRA.
- 2º control de lectura.

Modelos Generativos Profundos

Clase 8: Modelos autorregresivos y LLMs (parte IV)